

Estructuras de Datos

Clase 18 – Grafos: Recorridos y Búsqueda de caminos



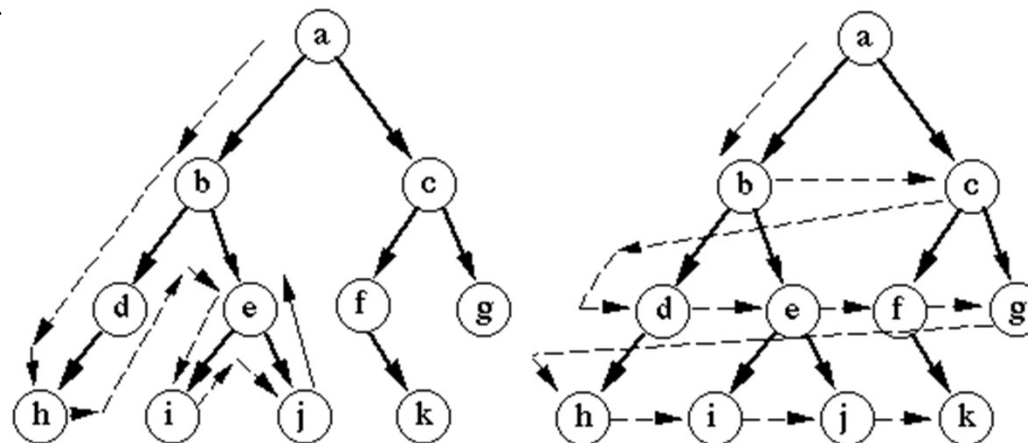
Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Recorridos de grafos

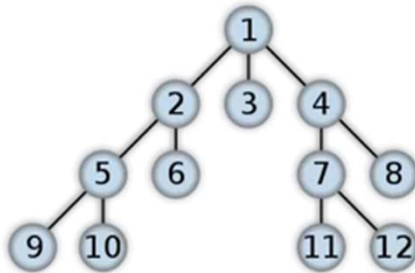
- En profundidad (Depth-First Search o DFS): (Equivale al recorrido pre orden en árboles + un testeo para no volver a recorrer un subgrafo ya explorado): a, b, d, h, e, i, j, c, f, k, g
- En anchura (Breadth-First Search o BFS): (Equivale al recorrido por niveles en árboles + un testeo para no volver a recorrer un subgrafo ya explorado): a, b, c, d, e, f, g, h, i, j, k



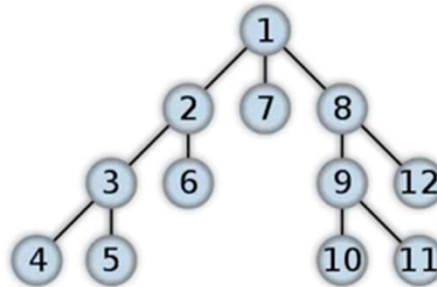
Depth-first search

Breadth-first search

BFS

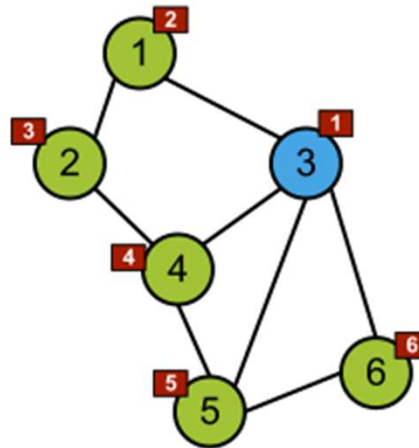
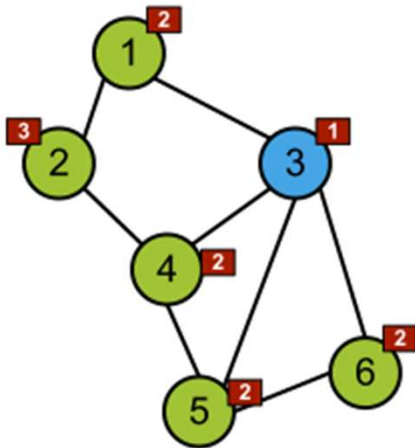


DFS



El número del
vértice
corresponde al
orden en el cual es
visitado por el
algoritmo
respectivo.

Breadth-First vs. Depth-First Search



Ambos recorridos salen
del 3. En rojo, está la
distancia al 3 en cantidad
de arcos de acuerdo al
recorrido.

BFS: 3, 1, 4, 5, 6, 2

DFS: 3, 1, 2, 4, 5, 6

Búsqueda en profundidad (DFS)

Algoritmo DFSShell(G : Grafo)

para cada vértice v de G hacer
 marcar v como no visitado
para cada vértice v de G hacer
 si v no está visitado entonces
 DFS(G, v)

Algoritmo DFS(G : Grafo; v : Vértice)

 procesamiento de v previo al recorrido
 marcar a v como visitado
 para cada vértice w adyacente a v en G hacer
 si w no está visitado entonces
 DFS(G, w)
 procesamiento de v posterior al recorrido

$$T_{\text{DFS}}(n,m) = O(n+m)$$

Análisis del tiempo de ejecución

- Sea un grafo $G=(V,A)$
- Sean $n = \text{cardinal}(V)$ y $m = \text{cardinal}(A)$
- Para simplificar el análisis, supongamos que el grafo es conexo.
- El algoritmo visita 1 vez cada vértice y en cada vértice i recorre su lista de adyacentes que mide A_i , entonces:

$$\begin{aligned} T(n, m) &= c_1 + \sum_{i=1}^n \left(c_2 + \sum_{j=1}^{A_i} c_3 \right) \\ &= c_1 + \sum_{i=1}^n (c_2 + A_i c_3) \\ &= c_1 + \sum_{i=1}^n c_2 + \sum_{i=1}^n A_i c_3 \\ &= c_1 + \sum_{i=1}^n c_2 + c_3 \sum_{i=1}^n A_i \\ &= c_1 + nc_2 + mc_3 \\ &\leq (n + m)c_1 + (n + m)c_2 + (n + m)c_3 \\ &= (n + m)(c_1 + c_2 + c_3) = O(n + m) \end{aligned}$$

Note que $m = O(n^2)$

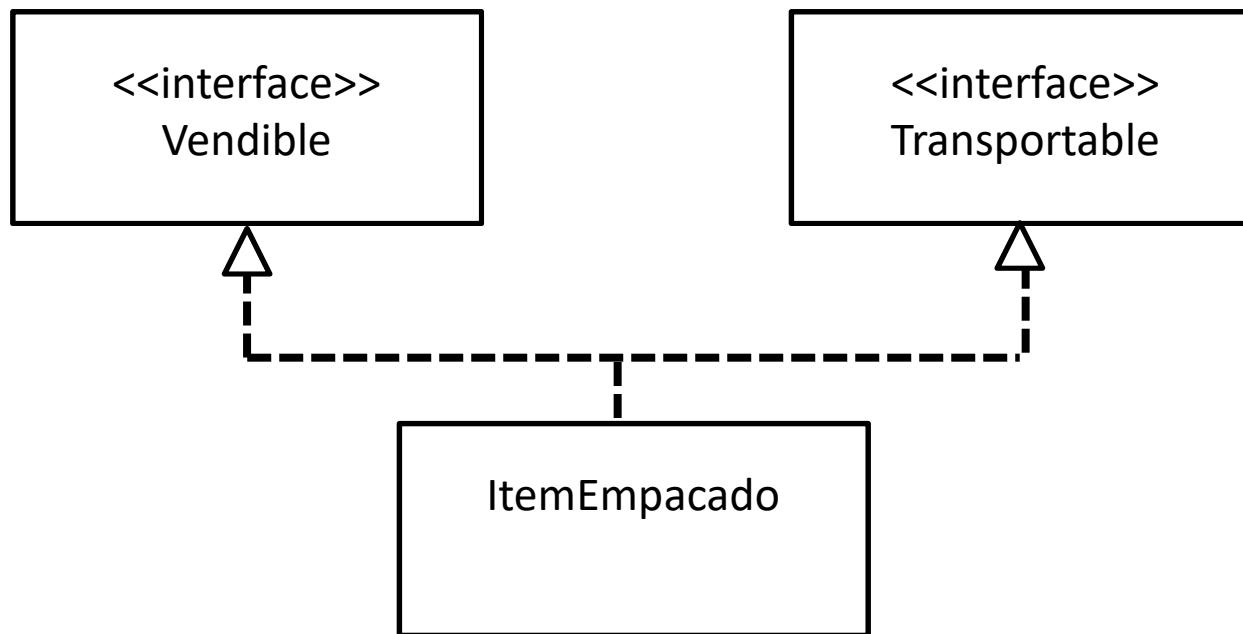
Nota: $\sum_{i=1}^n A_i = A_1 + A_2 + \dots + A_n = m$

Opciones para implementar las marcas de visitado

1. Usar un mapeo externo al grafo de $\text{Vertex} \langle V \rangle$ en Boolean
 - Ventaja: No hay que modificar el grafo que ya programamos.
 - Contra: El tiempo de marcar y desmarcar puede tender a $O(n)$
2. Agregar un boolean a la clase Vertice del grafo
 - Ventaja: Puedo garantizar que marcar y desmarcar funciona en $O(1)$
 - Contra: Por cada algoritmo que escribo tengo que ensuciar el grafo agregando atributos y operaciones
3. Decorar los vértices del grafo (opción de GT).
 - Ventaja: En forma abstracta mantengo toda la información del DFS y de futuros algoritmos
 - Contra: Hay que modificar lo que ya programamos

Repaso de clase 1: Clase que implementa más de una interfaz

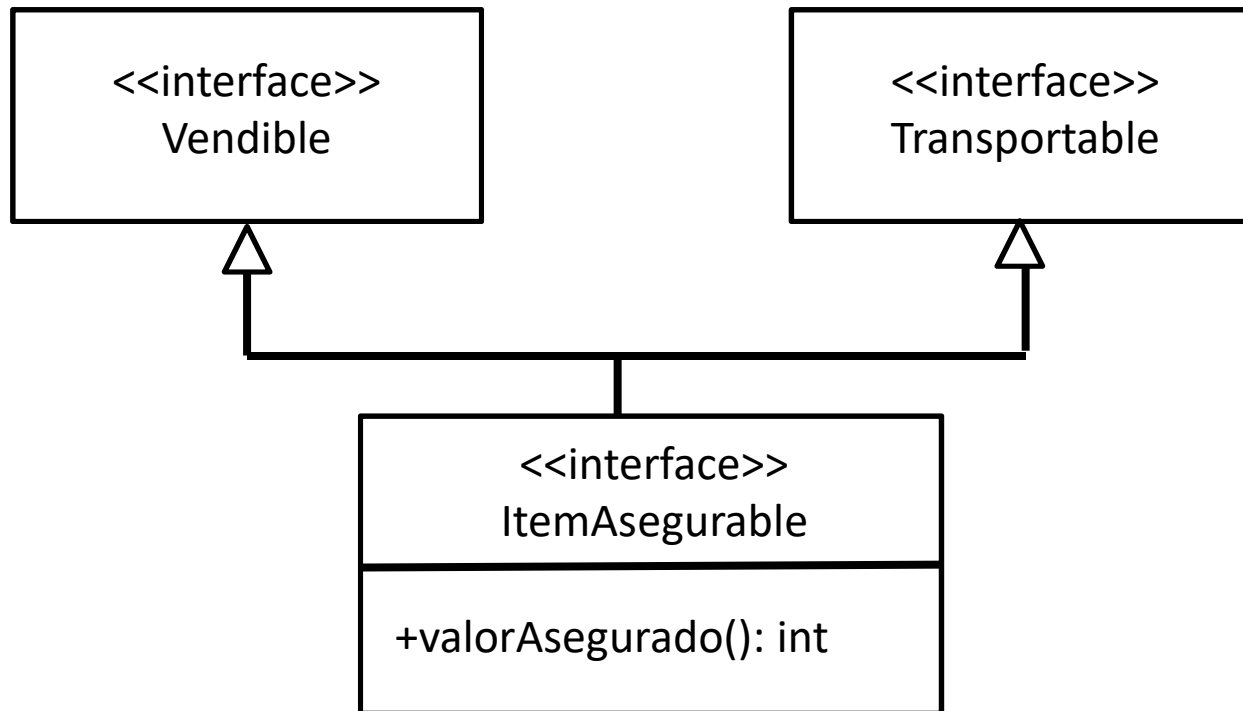
La clase *ItemEmpacado* implementa las interfaces *Vendible* y *Transportable*:



```
public class ItemEmpacado implements Vendible, Transportable {
    // Atributos:
    // Constructor:
    // Implementación de todos los métodos de ambas interfaces:
    // Implementación de métodos adicionales:
}
```

Repaso de clase 1: Herencia múltiple de interfaces

La interfaz *ItemAsegurable* extiende a las interfaces *Vendible* y *Transportable* y las extiende con el método *valorAsegurado*:



```
public interface ItemAsegurable extends Vendible, Transportable {  
    public int valorAsegurado();  
}
```

Opción 3: DFS con vértices decorados

Una posición decorada es una posición que además es un mapeo.

```
public interface DecorablePosition<E>
```

```
    extends Position<E>, Map<Object, Object> {
```

```
    ... sin métodos nuevos...
```

```
}
```

```
public interface Vertex<V> extends DecorablePosition<V> { }
```

```
public interface Edge<E> extends DecorablePosition<E> { }
```

Para implementar la clase *Vertice*, hago:

```
public class Vertice<V,E>
```

```
    implements Vertex<V>
```

```
    extends HashMap<Object, Object> {
```

```
        ... Idem a lo presentado en las clases anteriores ...
```

```
}
```

Vértices decorados

Si v es un `Vertex<Integer>`, entonces:

- `v.element()` retorna un entero que corresponde al rótulo de v
- `v.put( ESTADO, VISITADO)` permite anotar que v está visitado
- `v.get( ESTADO )` permite testear si v es un vértice visitado o no (que puede devolver `NO_VISITADO` o `null`).
- `ESTADO` y `VISITADO` son dos `Object` inicializados en la aplicación cliente del grafo.

Opción 3: DFS con vértices decorados

```
public class Aplicación {  
    private final Object VISITADO = new Object();  
    private final Object NOVISITADO = new Object() ;  
    private final Object ESTADO = new Object();  
  
    public <V,E> static void dfsShell( Graph<V,E> g ) {  
        for( Vertex<V> v : g.vertices() )  
            v.put( ESTADO, NOVISITADO );  
        for( Vertex<V> v : g.vertices() )  
            if( v.get( ESTADO ) == NOVISITADO )  
                dfs( g, v );  
    }  
    private <V,E> static void dfs( Graph<V,E> g, Vertex<V> v ) {  
        ...Presentado a continuación ...  
    }  
}
```

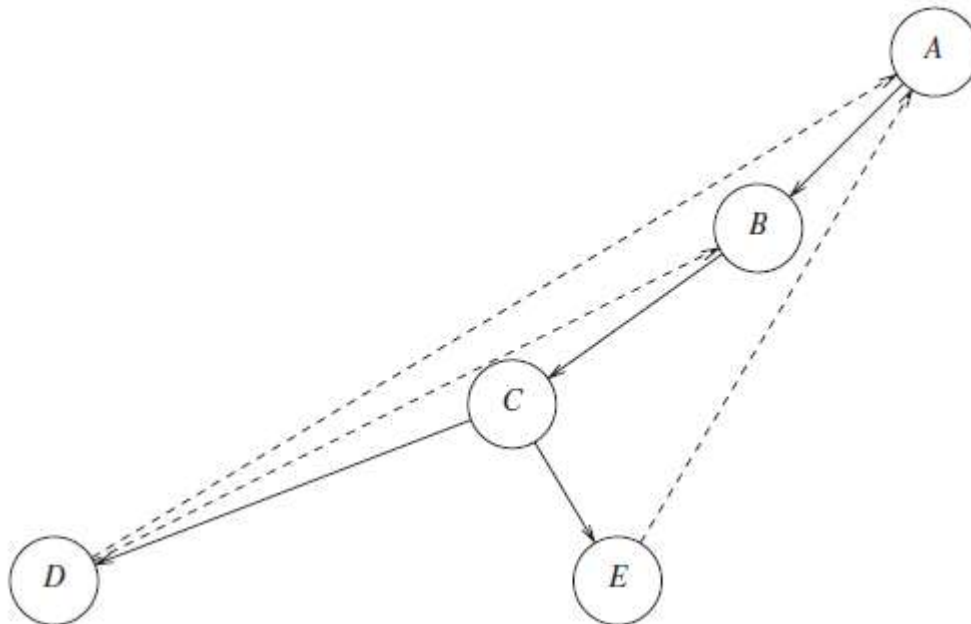
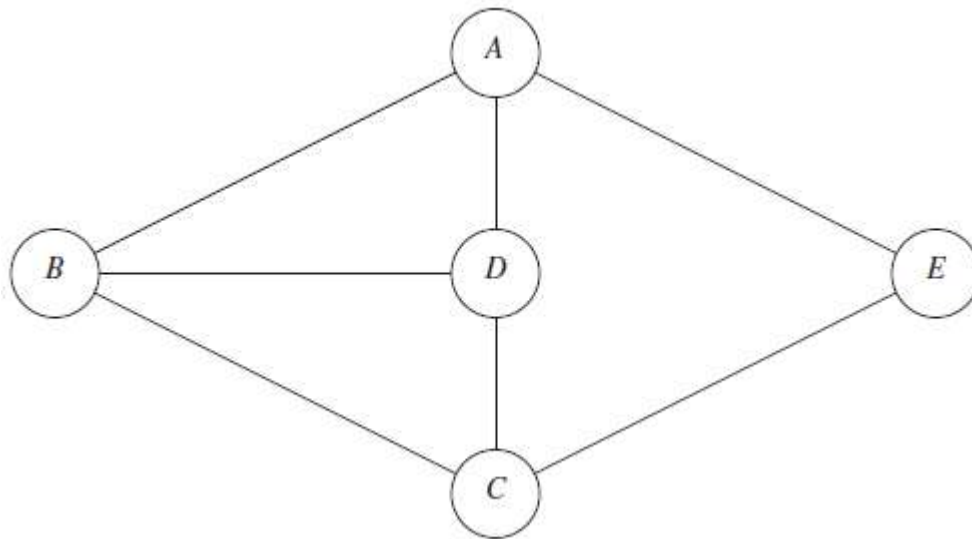
Opción 3: DFS con vértices decorados

```
private <V,E> static void dfs( Graph<V,E> g, Vertex<V> v ) {  
  
    // En este caso, el procesamiento de v es sólo imprimir su rótulo:  
    System.out.println( v.element() );  
  
    v.put( ESTADO, VISITADO );  
    Iterable<Edge<E>> adyacentes = g.incidentEdges( v ); // **  
    for( Edge<E> e : adyacentes ) {  
        Vertex<V> w = g.opposite( v, e );  
        if( w.get( ESTADO ) == NOVISITADO )  
            dfs( g, w );  
    }  
  
    // Aquí va el postprocesamiento de v si fuera necesario  
}
```

Nota: Si el grafo es dirigido cambiar ** por g.emergentEdges(v).

Bosque del DFS en grafos no dirigidos

- Las llamadas sucesivas a DFS desde DFSShell van *descubriendo* vértices que forman *árboles*.
- Estos árboles forman el bosque (o foresta) del DFS.
- Al orientar los arcos en la dirección en la que son explorados durante el recorrido, se distinguen:
 - Arcos de descubrimiento o árbol (discovery o tree edges): Arcos que llevan a vértices no visitados
 - Arcos de retroceso (back edges): Arcos que llevan a vértices que ya fueron visitados.



Ejemplo de árbol: Para este grafo no dirigido, debajo está el árbol que se genera al salir de A (los arcos llenos son los arcos *tree* y los árcos punteados son los *back*).

Si el grafo es no conexo, tendremos un árbol por cada llamada al DFS (i.e. para cada componente conexa). Los arcos back no son parte del árbol. Gráfico tomado de *Mark Allen Weiss, Data Structures and Algorithm Analysis in Java, Third Edition, Pearson, 2012.*

Bosque del DFS en grafos no dirigidos: Algoritmo

Algoritmo DFS(G : Grafo; v : Vertice)

Marcar a v como visitado

Para cada arco e en G.incidentEdges(v) hacer

si e no está visitado entonces

$w \leftarrow G.opposite(v, e)$

si w no está visitado entonces

 etiquetar a e como *arco de descubrimiento*

 DFS(G, w)

sino

 etiquetar a e como *arco de retroceso*

$T_{DFS}(n,m) = O(n+m)$ con lista de adyacencia

$T_{DFS}(n,m) = O(n^2)$ con matriz de adyacencia

Bosque del DFS con arcos decorados

```
public static void <V,E> DFS( Graph<V,E> G, Vertex<V> v, Object k )
{
    v.put( k, VISITADO );
    for( Edge<E> e : G.incidentEdges(v) ) {
        if( e.get( k ) == null ) {
            w = G.opposite(v, e );
            if( w.get( k ) == null ) {
                e.put( k, ARCO_DESCUBRIMIENTO );
                DFS( G, w, k );
            } else e.put( k, ARCO_RETROCESO );
        }
    }
}
```

$T_{DFS}(n,m) = O(n+m)$ asumiendo operaciones de mapeo en $O(1)$

Búsqueda en anchura (BFS)

- La búsqueda en anchura (BFS o Breadth First Search) es un algoritmo para recorrer o buscar elementos en un grafo.
- Se comienza eligiendo algún nodo como elemento raíz y se exploran todos los vecinos de este nodo.
- A continuación para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el grafo.

Búsqueda en anchura (Breadth-First Search o BFS)

Algoritmo BFSShell(G : Grafo)
 para cada vértice v de G hacer
 marcar v como no visitado
 para cada vértice v de G hacer
 si v no está visitado entonces
 BFS(G, v)

Algoritmo BFS(G: Grafo; v : Vertice)
 En próxima diapositiva ...

Búsqueda en anchura

El algoritmo hace un listado por niveles desde v pero teniendo en cuenta de no repetir la visita en vértices ya vistos, lo que se hace con el código marcado **así**:

Algoritmo BFS(G : Grafo; v : Vértice)

cola $\leftarrow$ new Cola()

cola.enqueue(v)

marcar a v como visitado

mientras not cola.isEmpty() hacer

u $\leftarrow$ cola.dequeue()

procesar a u

para cada vértice x adyacente a u hacer

si x no está visitado entonces

marcar a x como visitado

cola.enqueue(x)

$T_{\text{BFS}}(n,m) = O(n+m)$ (justificación en la próxima diapositiva)

Análisis del tiempo de ejecución

- Sea un grafo $G=(V,A)$
- Sean $n = \text{cardinal}(V)$ y $m = \text{cardinal}(A)$
- Para simplificar el análisis, supongamos que el grafo es conexo.
- Sea A_i la cantidad de adyacentes del vértice i :

$$\begin{aligned} T(n, m) &= c_1 + \sum_{i=1}^n \left(c_2 + \sum_{j=1}^{A_i} c_3 \right) \\ &= c_1 + \sum_{i=1}^n (c_2 + A_i c_3) \\ &= c_1 + \sum_{i=1}^n c_2 + \sum_{i=1}^n A_i c_3 \\ &= c_1 + nc_2 + mc_3 = O(n + m) \end{aligned}$$

```
Algoritmo BFS( G : Grafo; v : Vértice )
cola ← new Cola()
cola.enqueue( v )
marcar a v como visitado
mientras not cola.isEmpty() hacer
    u ← cola.dequeue()
    procesar a u
    para cada vértice x adyacente a u hacer
        si x no está visitado entonces
            marcar a x como visitado
            cola.enqueue( x )
```

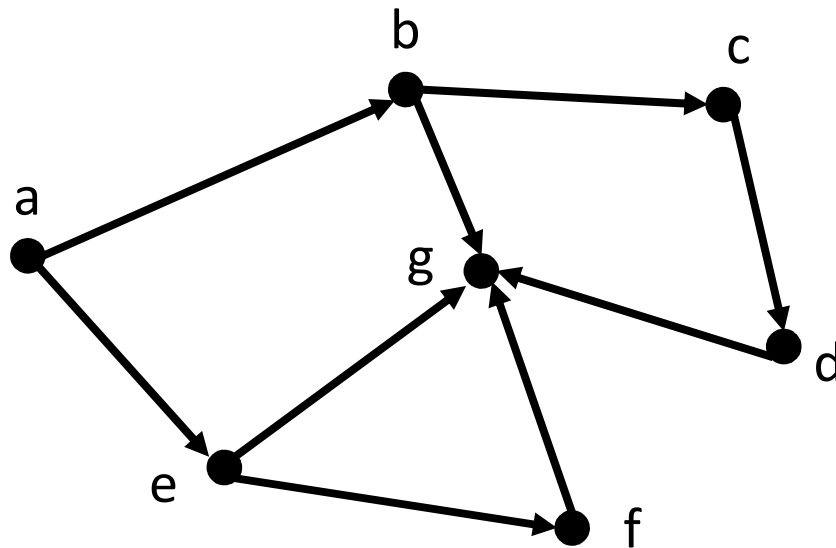
Note que $m = O(n^2)$

Aplicaciones del DFS para grafos no dirigidos en $O(n+m)$

- Testear si G es conexo (todos los vértices quedan visitados si y sólo si el grafo es conexo)
- Calcular un árbol abarcador si G es conexo (formado por los vértices de G y por sus arcos tree)
- Calcular las componentes conexas (por cada iteración de DFSShell incremento un contador indicando el número de componente conexa y con ese contador etiqueto los vértices de cada componente)
- Encontrar un camino entre dos nodos (clase siguiente)
- Encontrar un ciclo (clase siguiente)

Definiciones

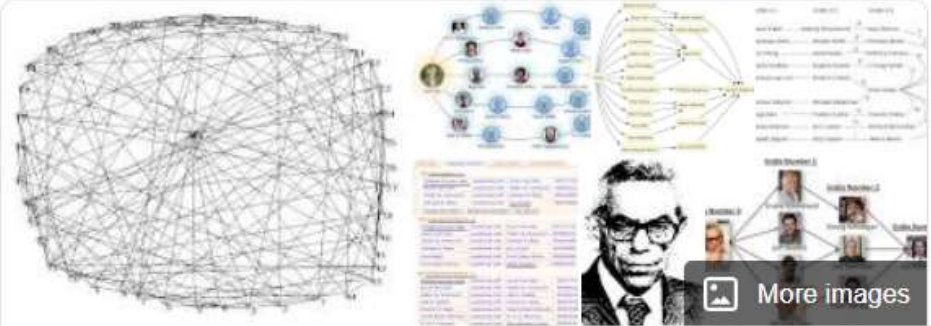
- Camino: Un camino de v a w es una secuencia alternante de vértices y arcos tales que empieza en v y termina w y cada arco es incidente en sus vértices predecesor y sucesor.



a, b, c, d, g es un camino.
 a, e, g es otro camino.

Aplicación de camino: Erdős number

El *número de Erdős* de un investigador X es la longitud del camino de X a Paul Erdős en el grafo de coautores científicos. Paul Erdős fue el matemático más prolífico de la historia (publicó alrededor de 1500 artículos).



Erdős number

The Erdős number describes the "collaborative distance" between mathematician Paul Erdős and another person, as measured by authorship of mathematical papers. The same principle has been applied in other fields where a particular individual has collaborated with a large and broad number of peers. [Wikipedia](#)

csauthors.net Distance computation (NEW) Credits Type an author

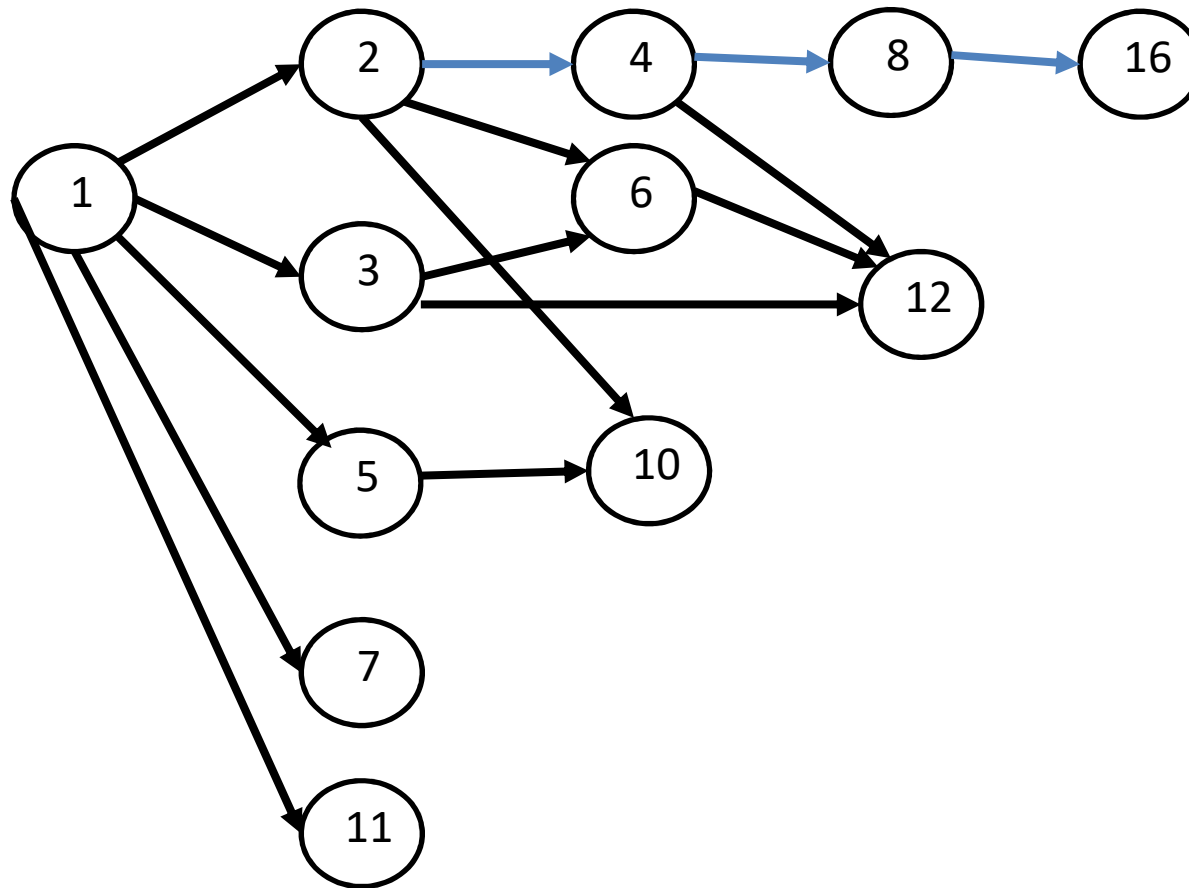
Co-authorship distance computation

Find the path between two authors:

Paul Erdős Sergio Alejandro Gómez

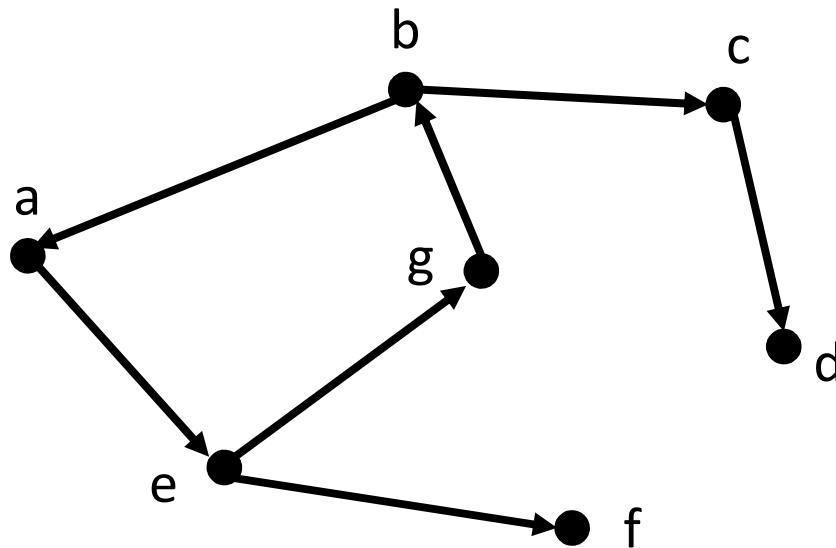
Paul Erdős
co-authored 7 papers with [Fan Chung Graham](#)
co-authored 1 paper with [Alessandro Vespignani](#)
co-authored 4 papers with [Ana Gabriela Maguitman](#)
co-authored 24 papers with [Carlos Iván Chesñevar](#)
co-authored 6 papers with [Sergio Alejandro Gómez](#)
distance = 5

Ejemplo de camino en grafo dirigido: Como hay un camino de 2 a 16, entonces inferimos que 2 divide a 16.



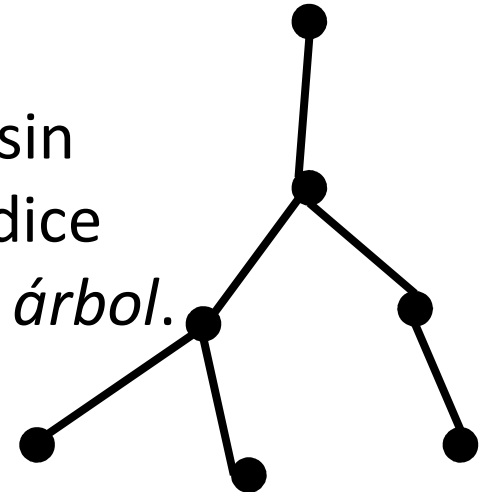
Definiciones

- Ciclo (simple): Un camino que comienza y termina en el mismo vértice (sin repetir arcos).
- Ciclo (camino) dirigido: Un ciclo (camino) donde las aristas tienen dirección y son recorridas en su dirección.



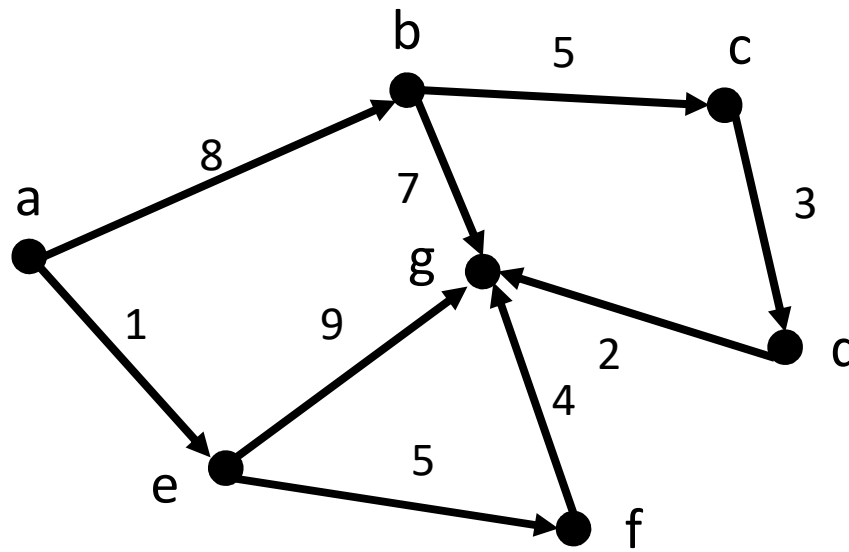
a, e, g, b, a es un ciclo dirigido.

Un grafo sin ciclos se dice acíclico o *árbol*.



Definiciones

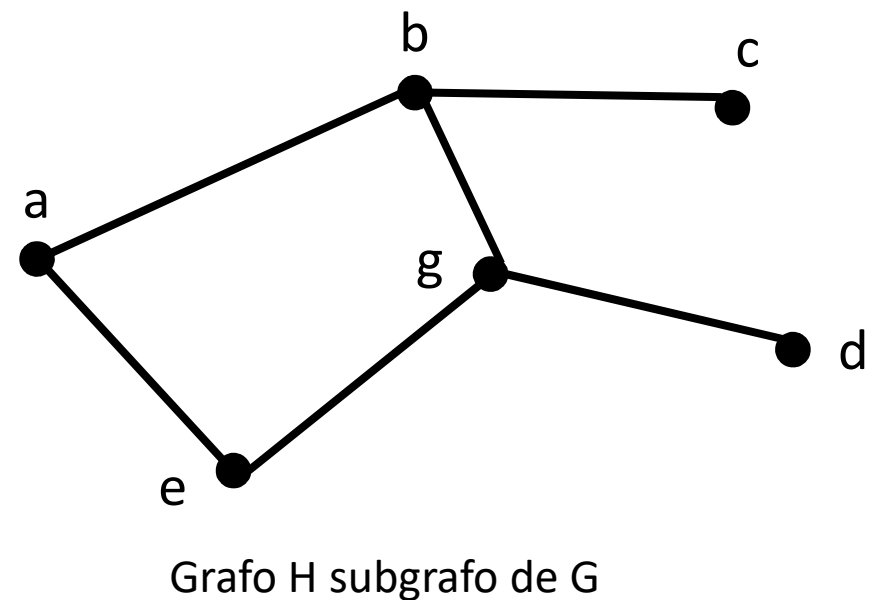
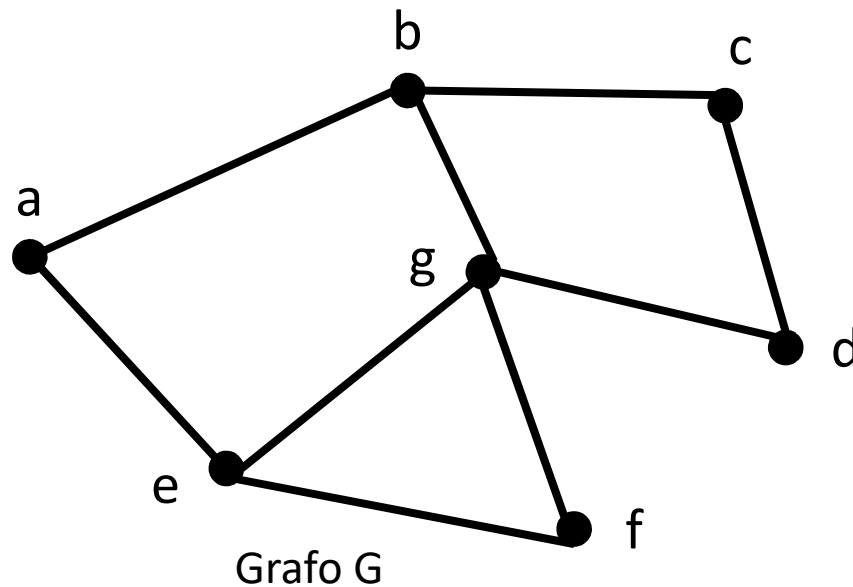
- Costo de un camino (ciclo): Suma de los pesos de los arcos que forman el camino (ciclo)



- Ejemplo: El costo del camino “a, b, c, d, g” es $8+5+3+2=18$.

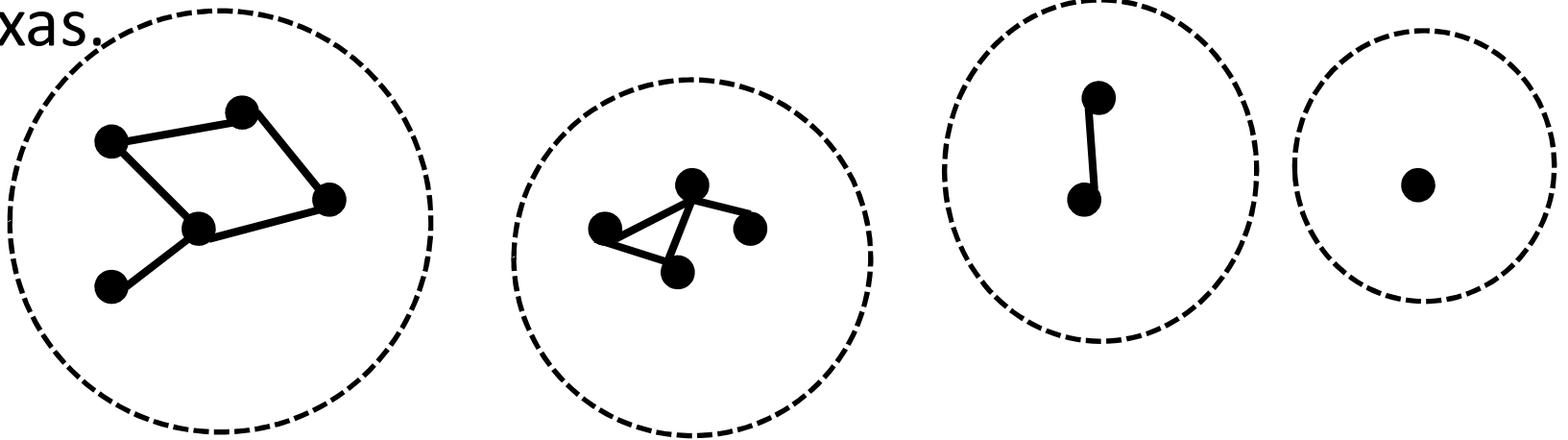
Definiciones

- Subgrafo: Un subgrafo H de un grafo G es un grafo donde los vértices de H son un subconjunto de los vértices de G y los arcos de H son un subconjunto de los arcos de G .



Definiciones

- Grafo conexo: Un grafo G es conexo si para dos vértices cualquiera de G hay un camino entre ellos.
- Componentes conexas: Si un grafo no es conexo, sus subgrafos maximales conexos se llaman componentes conexas.



Este grafo no es conexo porque tiene 4 componentes conexas.

Nota: En grafos dirigidos se habla de “grafo fuertemente conexo” y componente fuertemente conexas porque se pide que los caminos entre vertices sean dirigidos.

Aplicaciones del BFS

Idem DFS y además:

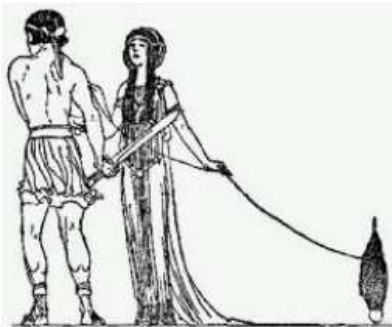
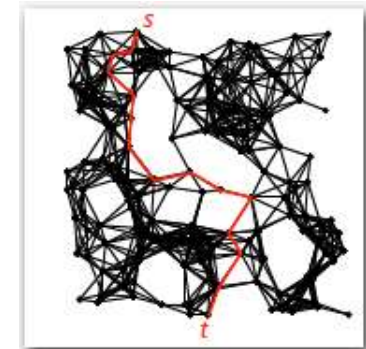
- Hallar el camino más corto (en cantidad de arcos) entre dos vértices (en $O(n+m)$) (en clase siguiente).

Algoritmos para encontrar caminos en digrafos pesados con números reales

- DFS pinchado (st-path search): Permite encontrar un camino entre dos vértices s y t
- BFS para hallar camino con cantidad mínima de arcos.
- DFS con backtracking, marca y desmarca: Permite encontrar un camino de costo mínimo entre dos vértices s y t
- Dijkstra: Permite hallar todos caminos de costo mínimo entre un vértice a y todos los otros vértices
- Floyd: Permite hallar el camino de costo mínimo entre cada par de vértices s y t .

St-path search

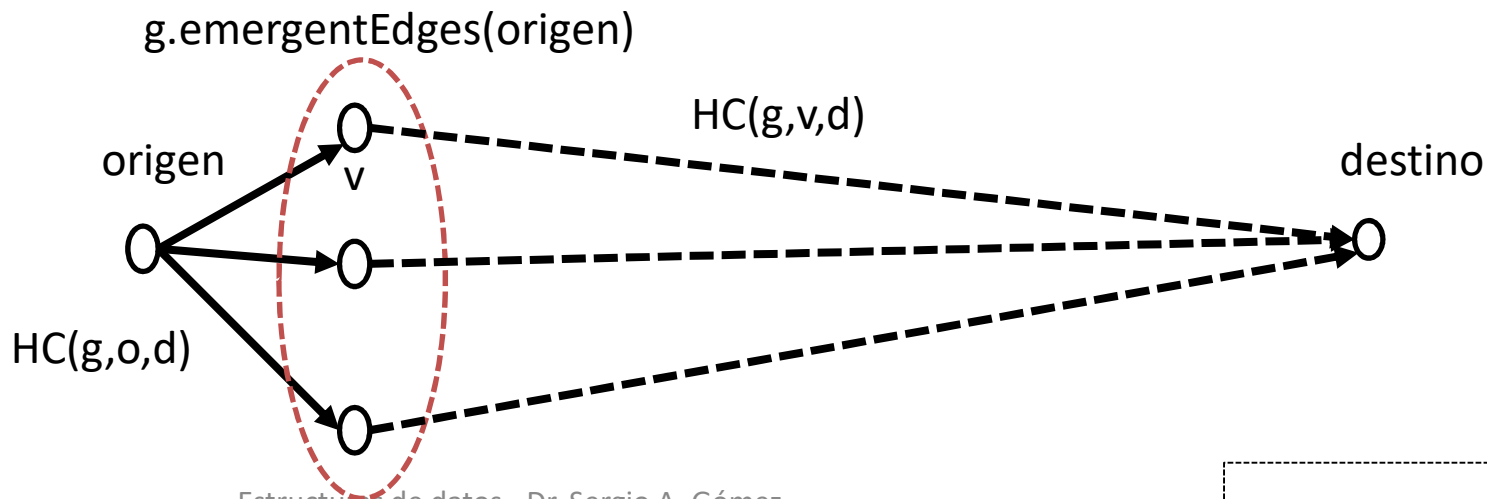
- Problema: Buscar un camino en g de s a t .
- S = source = origen
- T = target = destino
- Estrategia: Explorar el grafo en DFS manteniendo el camino explorado y marcando los vértices visitados para no volver a explorar porciones del grafo ya exploradas.



El grafo es un laberinto y almacenar el camino hace las veces del hilo de Ariadna para que Teseo no se pierda al entrar a buscar al Minotauro. (Imágenes de la web)

st-path search: Hallar camino de o a d en g

- Planteo recursivo:
- $\text{HallarCamino}(g: \text{Grafo}, o: \text{Vertice}, d: \text{Vertice}) : \text{Boolean}$
- CB: Si $o=d \Rightarrow \text{HallarCamino}(g,o,d) = \text{true}$
- CB: Si $o \neq d$ y o no tiene adyacentes $\Rightarrow \text{HallarCamino}(g,o,d) = \text{false}$
- CR: Si $o \neq d$ y o tiene adyacentes $\Rightarrow \text{HallarCamino}(g,o,d) = (\exists v \in g.\text{emergentEdges}(o)).(\text{HallarCamino}(g,v,d) = \text{true})$



DFS pinchado (s-t path search)

Permite hallar un camino en el digrafo G entre Origen y Destino y retorna el camino hallado en Camino (el cual es una lista vacía al principio).

Si encontró un camino, retorna verdadero, en caso contrario retorna falso.

Algoritmo HallarCamino(G, origen, destino, camino) : boolean

origen.put(Estado, Visitado) { *marco a origen como visitado* }

camino.addLast(origen) { *encolo a origen en el camino* }

Si origen = destino **entonces** { CB positivo }

retornar verdadero { *encontré el camino* }

Sino {CB negativo y CR }

para cada adyacente v de origen en G **hacer**

 si v.get(Estado) = NoVisitado **entonces**

 encontre $\leftarrow$ HallarCamino(G, v, destino, camino)

si encontre **entonces retornar** verdadero { *¡Retornar rompe el para!* }

{ *Exploré todos los adyacentes y no encontré el camino, ú origen no tiene adyacentes
=> hago backtracking: borro a origen del camino y retorno falso.* }

camino.remove(camino.last()) { Backtracking: elimino ultimo vertice del camino }

retornar falso

Estructuras de datos - Dr. Sergio A. Gómez

Complejidad:

$T(n,m) = O(n+m)$

Ejercicio: Codificar en Java

Estrategia para hallar ciclos

{ Encuentra un ciclo que contenga a v, buscando caminos desde los adyacentes de v (que se llaman w) hacia v. }

Algoritmo HallarCiclo(G, v) : Lista

 encontre $\leftarrow$ falso

 mientras hay adyacentes para considerar y no encontre hacer

 w $\leftarrow$ siguiente adyacente de v en G

 encontre $\leftarrow$ HallarCamino(G, w, v, camino)

 finmientras

 ciclo $\leftarrow$ new Lista()

 si encontre entonces

 ciclo.addLast(v)

 para cada vertice x de camino hacer

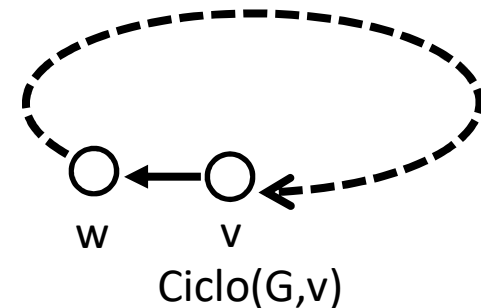
 ciclo.addLast(x)

 finpara

 finsi

 retornar ciclo

HallarCamino(G,w,v,camino)



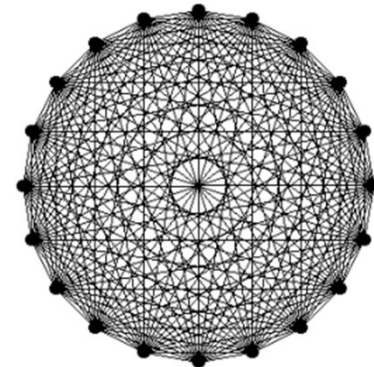
$$T_{\text{hallarciclo}}(n,m) = O(\text{grado}(v) * (n+m) + n)$$

DFS con marca y desmarca

Dado un digrafo pesado con números reales, permite hallar un camino de costo mínimo entre dos vértices origen y destino computando el camino y su costo (entendido como la suma de los pesos de los arcos).

El tiempo de ejecución para un grafo que tiene todos los arcos entre cada par de nodos es $O(n!)$ (en la práctica esto es mucho menos, porque $n!$ se da en el peor caso que es cuando todos los vértices están conectados con todos los otros vértices)

Este caso se da cuando el grafo es completo.



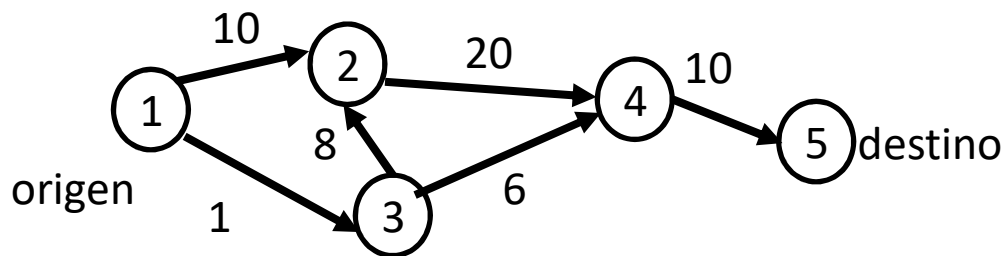
Estrategia para buscar camino de costo mínimo

La estrategia consiste de generar por fuerza bruta todos los caminos posibles de origen a destino con su costo.

Hago un proceso de mínimo: Cada vez que encuentro un nuevo camino de origen a destino, me fijo si el costo del nuevo camino hallado es mejor que el mejor que tengo hasta este momento. Si es así, lo actualizo.

Para generar todos los caminos posibles debo:

- (1) Marcar vértices como visitados para no entrar en bucles infinitos si el grafo tiene ciclos dirigidos.
- (2) Desmarcar vértices al hacer backtracking para permitir que un vértice pueda pertenecer a más de un camino posible.



*El camino 1-2-4-5 tiene costo
 $10+20+10=40$*

*El camino 1-3-2-4-5 tiene costo
 $1+8+20+10=39$*

*El camino 1-3-4-5 tienen costo
 $1+6+10=17$*

Para explorar el segundo y tercer caminos es necesario desmarcar los vértices ya visitados al generar el primer camino.

Al comienzo, *camino_actual* es una lista vacía y *costo_camino_actual* es 0.

El costo del camino es la suma de los pesos de los arcos del camino.

Algoritmo HallarCaminoMinimo(*G*, origen, destino, camino_actual,
costo_camino_actual, camino_minimo, costo_camino_minimo)

origen.put(Estado, Visitado) { marco a origen como visitado }

Camino_actual.addLast(origen) { encolo a origen en el camino_actual }

Si origen = destino **entonces** { llegué de nuevo a destino }

Si costo_camino_actual < costo_camino_minimo **entonces**

 { encontré un camino mejor => actualizo el mejor camino hasta el momento }

 camino_minimo ← camino_actual.clone_profundidad()

 costo_camino_minimo ← costo_camino_actual

Sino { origen <> destino => para cada adyacente de origen, veo si puedo llegar a destino }

para cada adyacente *v* de origen en *G* **hacer**

si *v*.get(Estado) = NoVisitado **entonces**

 HallarCaminoMinimo(*G*, *v*, destino, camino_actual,

 costo_camino_actual + peso_arco(origen,*v*), camino_minimo,
 costo_camino_minimo)

camino_actual.remove(camino_actual.last()) { *backtracking: saco a origen del camino* }

origen.put(Estado, NoVisitado) { desmarco a origen así se puede reusar para otros caminos }

Mantenimiento del costo mínimo

Por lo tanto se recomienda usar una clase:

```
class Solucion {  
    float costo;  
    ... setters y getters para el costo...  
}
```

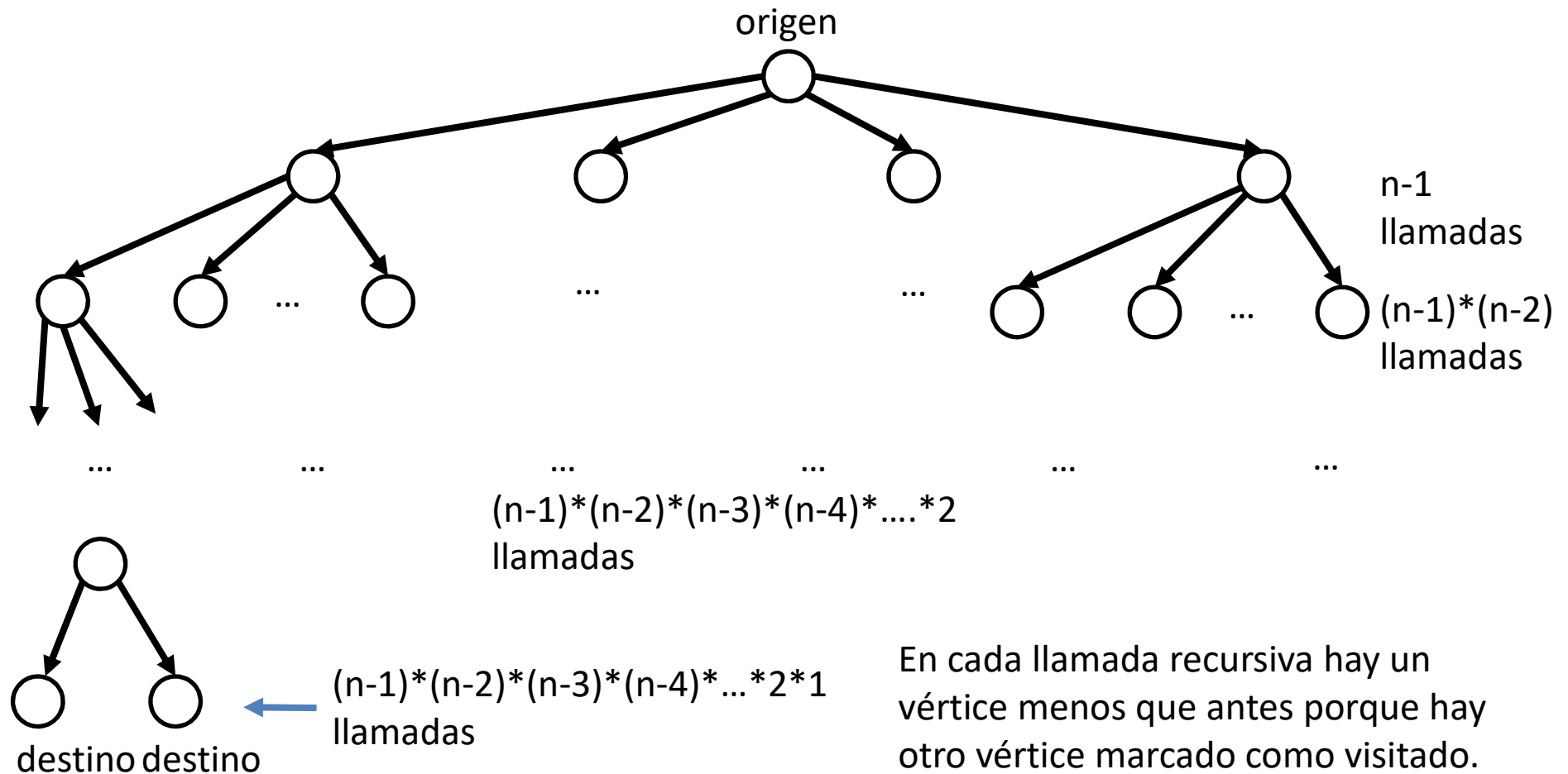
O mejor aún que permite almacenar cualquier tipo A como solución:

```
class Solucion<A> {  
    A estado;  
    ... setters y getters para el estado ...  
}
```

¿Por qué $T(n)=O((n-1)!)$?

Supongamos que el grafo de entrada tiene n vértices y cada vértice está conectado con los otros $n-1$ vértices.

¿Cómo es el espacio de búsqueda del problema?



BFS para hallar caminos

// Retorna true si hay camino de s a t en G.

// Previo es un mapeo tal que previo.get(v) retorna el nodo previo de v

// en el camino de s a v.

BFSSearch(G : Graph<V,E>, s : Vertex<V>, t : Vertex<V>;

 previo: Map<Vertex<V>, Vertex<V>>) : boolean

encolar s en Q; marcar s

while Q no está vacía Do

 desencolar x de Q

 if x = t then return true

 for cada adyacente v de x do

 if v no marcado then

 encolar v en Q

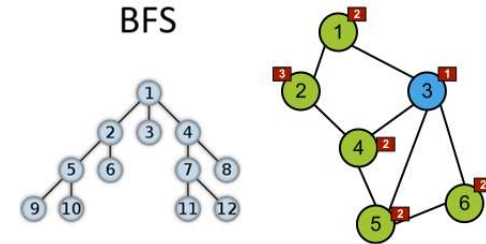
 marcar v

 previo.put(v,x)

Return false

Nota: Encuentra el camino más corto en cantidad de arcos entre s y t

$T(n,m) = O(n+m)$



NOTA: Es la misma estrategia que para buscar en un árbol.
La diferencia con un árbol es que en el árbol, el previo de v solo puede ser el padre de v mientras que acá hay que anotarlo explícitamente en un mapeo ya que le previo podría ser cualquiera de los predecesores de v.

Recuperación del camino

Recuperar(s, t, previo) : Lista

$x \leftarrow t$

Crear Lista L

Mientras $x \neq \text{null}$

 L.addFirst(x)

$x \leftarrow \text{previo.get}(x)$

Fin mientras

Retornar L

$T(n) = O(n)$ cuando con n la longitud del camino (que se puede acotar con la cantidad de vértices del grafo)

Algoritmo de Dijkstra

- Suponga un digrafo G tal que cada arco tiene costo no negativo, Dijkstra computa los caminos de costo mínimo desde un vértice a a todos los otros vértices de G .
- El vértice a se conoce como la *fuentes*.
- El costo del camino es la suma de los pesos de los arcos del camino.
- El algoritmo mantiene un conjunto S con los vértices cuyo camino con la distancia más corta es conocida.
- S inicialmente está vacío.
- En cada paso se agrega a S el vértice u cuya distancia a la fuente es tan cercana como es posible.
- El algoritmo termina cuando S contiene todos los vértices.
- La salida del algoritmo son dos mapeos $D : V \rightarrow \text{Float}$ y $P : V \rightarrow V$ tal que:
 - $D(v)$ es la distancia a v desde la fuente
 - $P(v)$ es el vértice anterior a v en el camino desde la fuente a v .

Pseudocódigo del Algoritmo de Dijkstra

Algoritmo Dijkstra

Entrada: G : digrafo simple conexo con todos los pesos positivos y a : Vertice

{ G tiene vértices $a=v_0, v_1, \dots, v_n$ y pesos $w(v_i, v_j)$ donde $w(v_i, v_j)=\infty$ si (v_i, v_j) no es un arco en G }

Salida: D : mapeo de vértice en float y P : mapeo de vértice en vértice

```
for  $i := 1$  to  $n$  do begin    { para cada vértice  $i$  }
     $D(i) := \infty$                 { Asumo que la distancia de la fuente  $a$  a  $i$  es infinita }
     $P(i) := \text{null}$               { Asumo que no hay anterior a  $i$  en el camino de  $a$  a  $i$  }
end
 $D(a) := 0$                       { La distancia de  $a$  a  $a$  es 0 }
 $S := \emptyset$                   { No hay vértices procesados }
for  $i := 1$  to  $n$  do begin    { repetir  $n$  veces }
     $u :=$  un vértice no en  $S$  con  $D(u)$  mínimo { elegir el vértice  $u$  más cercano a  $a$  }
     $S := S \cup \{u\}$               { marcar a  $u$  como procesado }
    for cada vértice  $v$  adyacente a  $u$  y que no está en  $S$  do
        if  $D(u) + w(u, v) < D(v)$  then begin { hacer edge relaxation }
             $D(v) := D(u) + w(u, v)$       { anotar nueva distancia de  $a$  a  $v$  }
             $P(v) := u$                   { anotar que el anterior de  $v$  es  $u$  en el camino }
        end
    end
end
return  $(P, D)$                   { retornar resultados }
```

Complejidad Algorítmica del Algoritmo de Dijkstra

Algoritmo Dijkstra

Entrada: G : digrafo simple conexo con todos los pesos positivos y a : Vertice

$\{ G \text{ tiene vértices } a=v_0, v_1, \dots, v_n \text{ y pesos } w(v_i, v_j) \text{ donde } w(v_i, v_j)=\infty \text{ si } (v_i, v_j) \text{ no es un arco en } G \}$

Salida: D : mapeo de vértice en float y P : mapeo de vértice en vértice

for $i := 1$ **to** n **do begin**

$D(i) := \infty$

$P(i) := \text{null}$

end

$D(a) := 0$

$S := \emptyset$

for $i := 1$ **to** n **do begin**

$u :=$ un vértice no en S con $D(u)$ mínimo

$S := S \cup \{ u \}$

for cada vértice v adyacente a u y que no está en S **do**

if $D(u) + w(u, v) < D(v)$ **then begin**

$D(v) := D(u) + w(u, v)$

$P(v) := u$

end

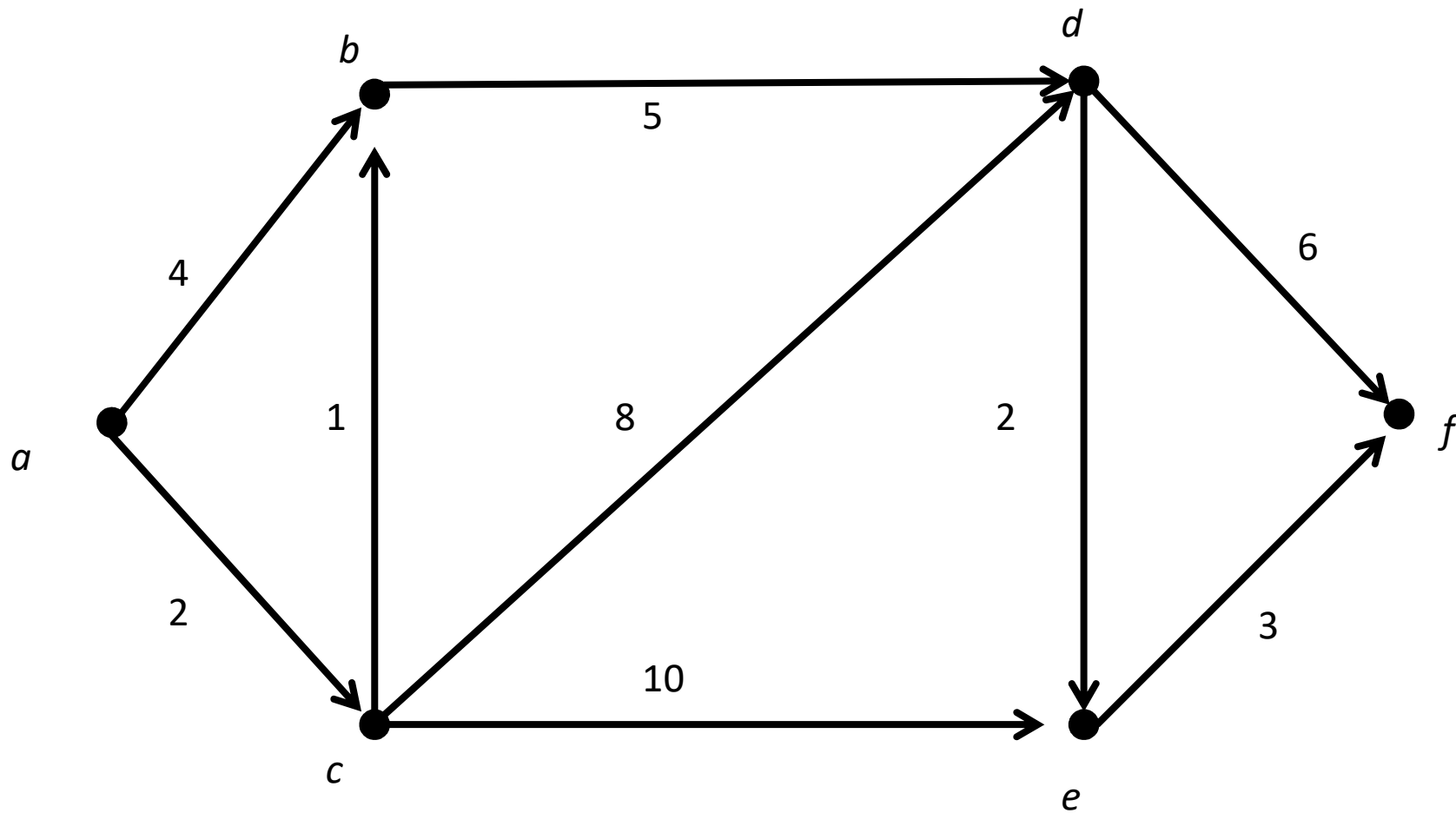
end

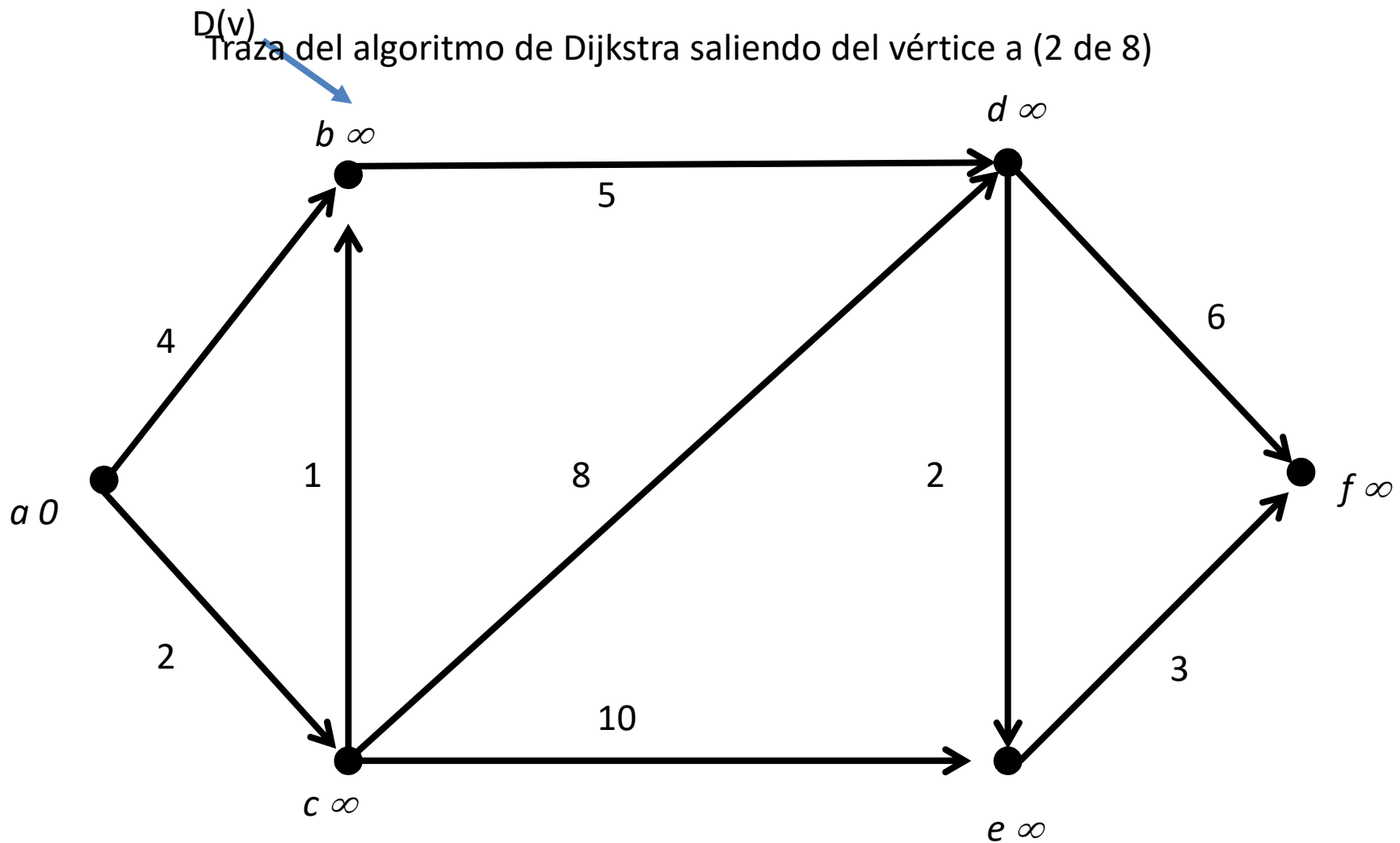
return (P, D)

$T_{\text{Dijkstra}}(n) = O(n^2)$ si G tiene n vértices y está representado con matriz

$T_{\text{Dijkstra}}(n, m) = O((m + n)\log(n))$ (son m actualizaciones a una cola con prioridades adaptable de n elementos), si G está representado con lista de adyacencias y get y put del mapeo tienen $O(1)$.

Traza del algoritmo de Dijkstra saliendo del vértice a (1 de 8)

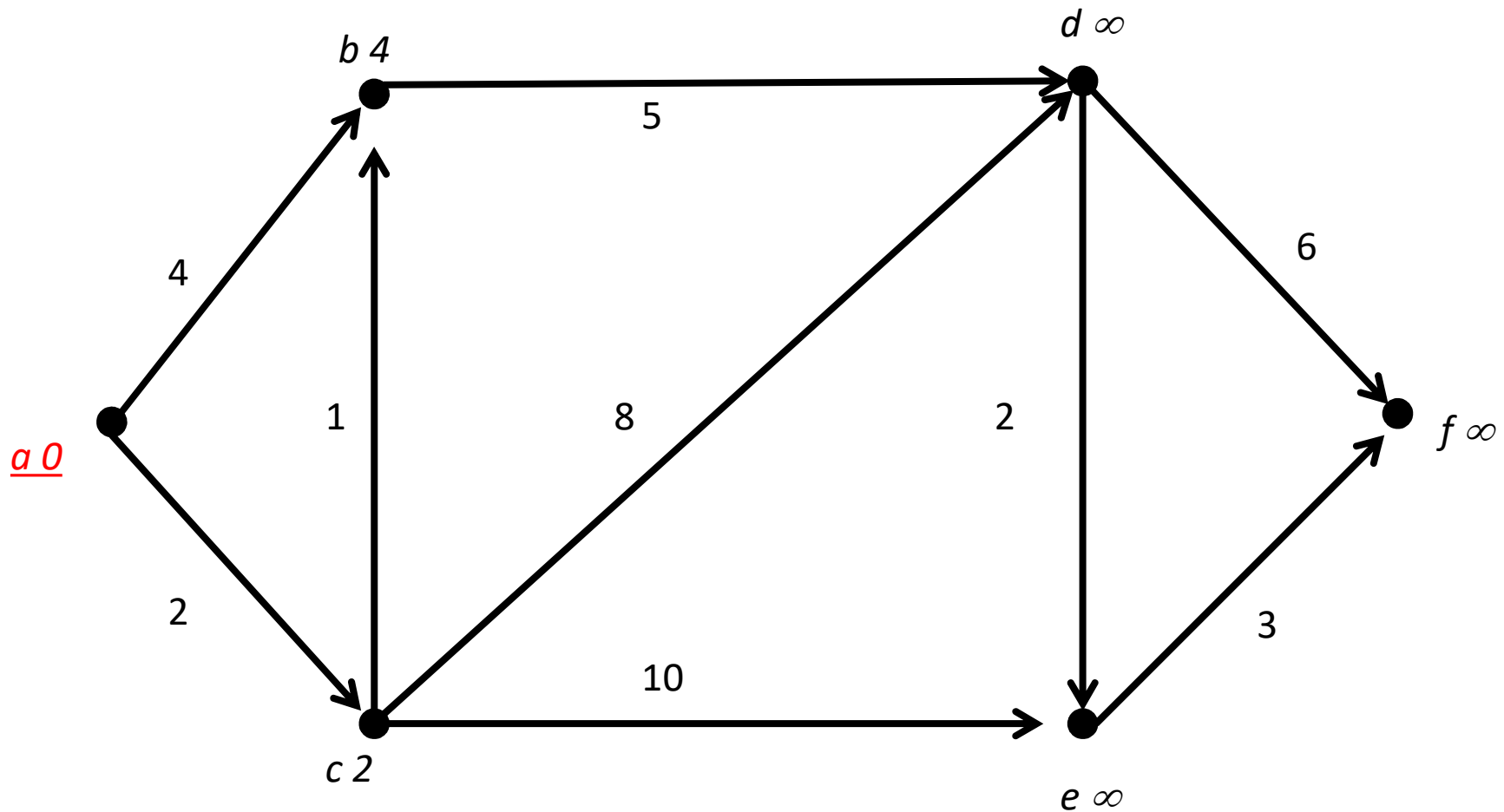




$S = \emptyset$

El primer vértice u que encuentra el algoritmo es a puesto que $D(a) = 0$ y todos los otros $D(v) = \infty$.

Traza del algoritmo de Dijkstra saliendo del vértice a (3 de 8)

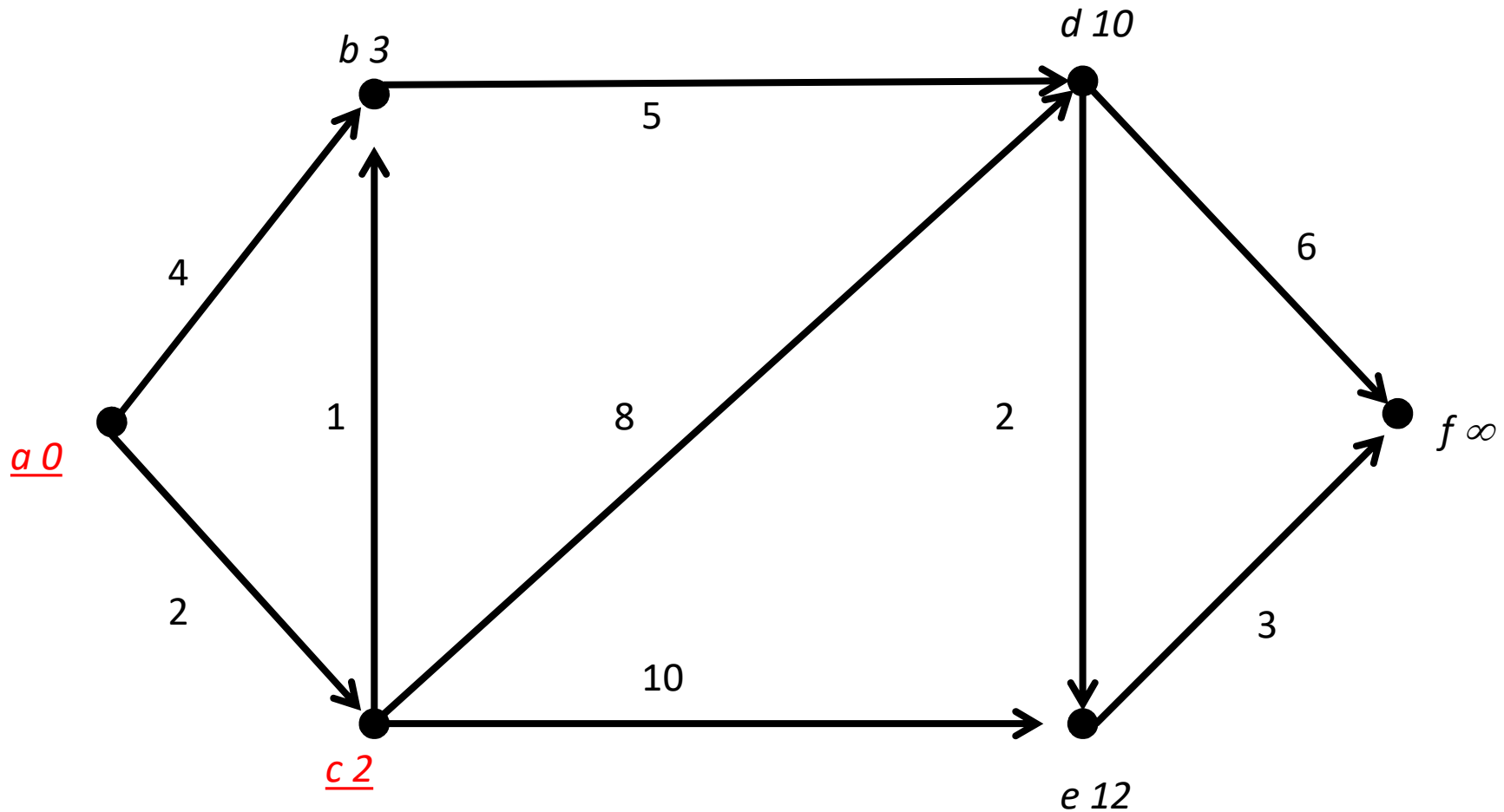


$u = a$

Actualiza a b y a c con $D(b) = 4$ y $D(c) = 2$. $S = \{a\}$. $P(b) = a$ y $P(c) = a$.

En la siguiente iteración el mínimo de D será c .

Traza del algoritmo de Dijkstra saliendo del vértice a (4 de 8)



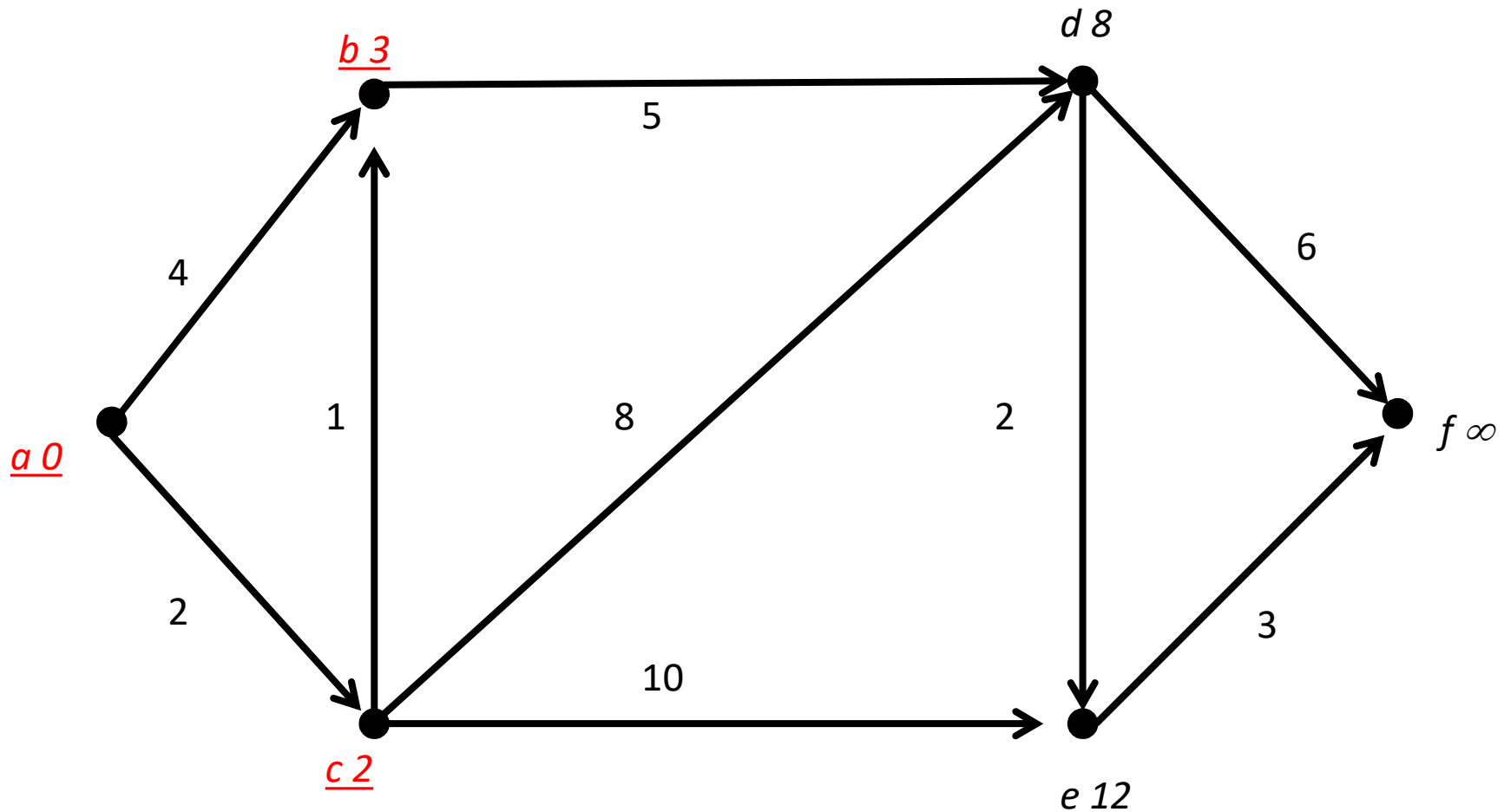
$u = c$

Actualiza a b , d y e con $D(b) = 3$, $D(d) = 10$ y $D(e) = 12$.

$S = \{a, c\}$. $P(b) = a$, $P(d) = c$, $P(e) = c$.

En la siguiente iteración el mínimo de D será b .

Traza del algoritmo de Dijkstra saliendo del vértice a (5 de 8)



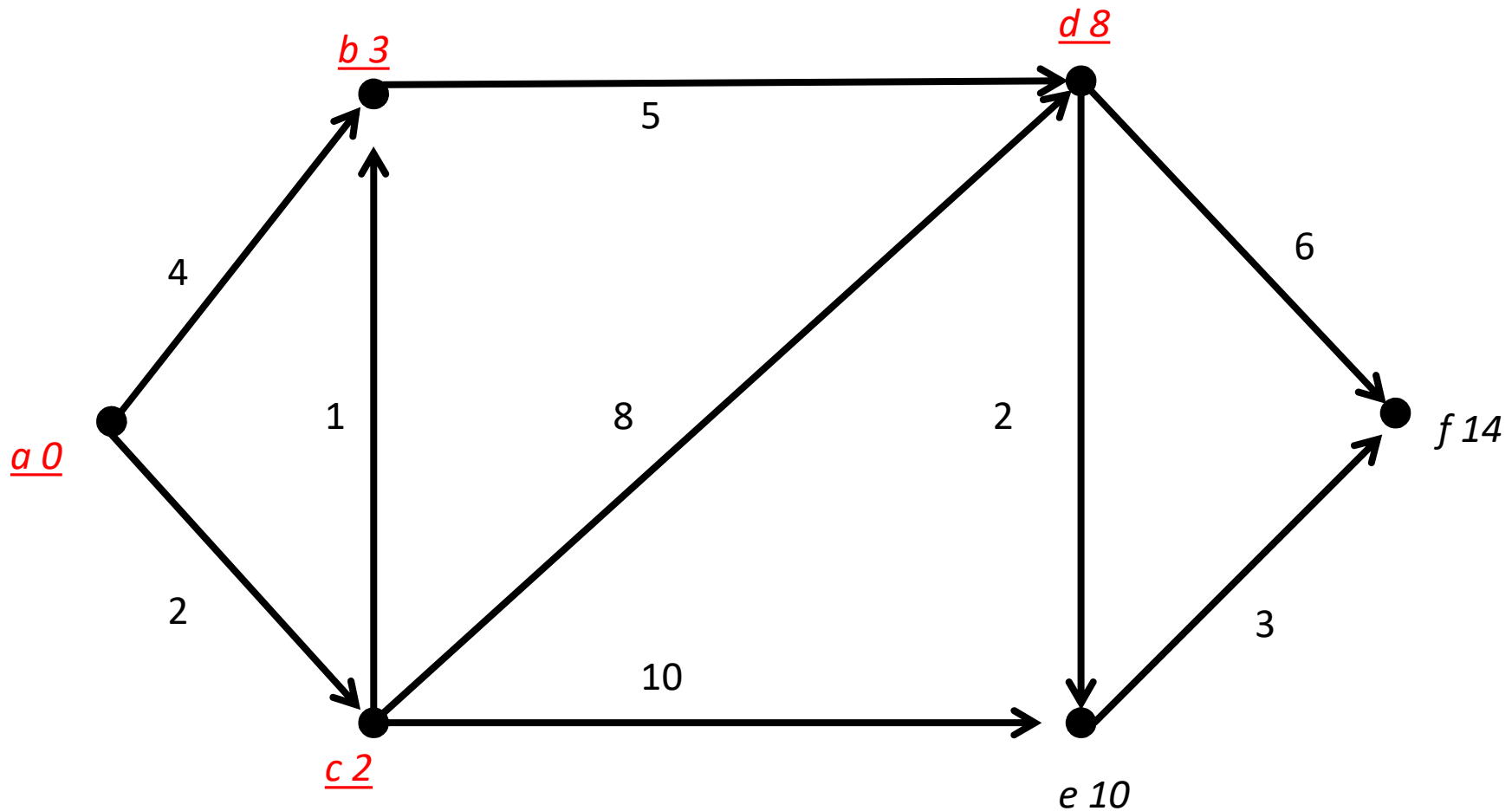
$u = b$

Actualiza a d con $D(d) = 8$,

$S = \{a, b, c\}$. $P(d) = 3$

En la siguiente iteración el mínimo de D será d .

Traza del algoritmo de Dijkstra saliendo del vértice a (6 de 8)



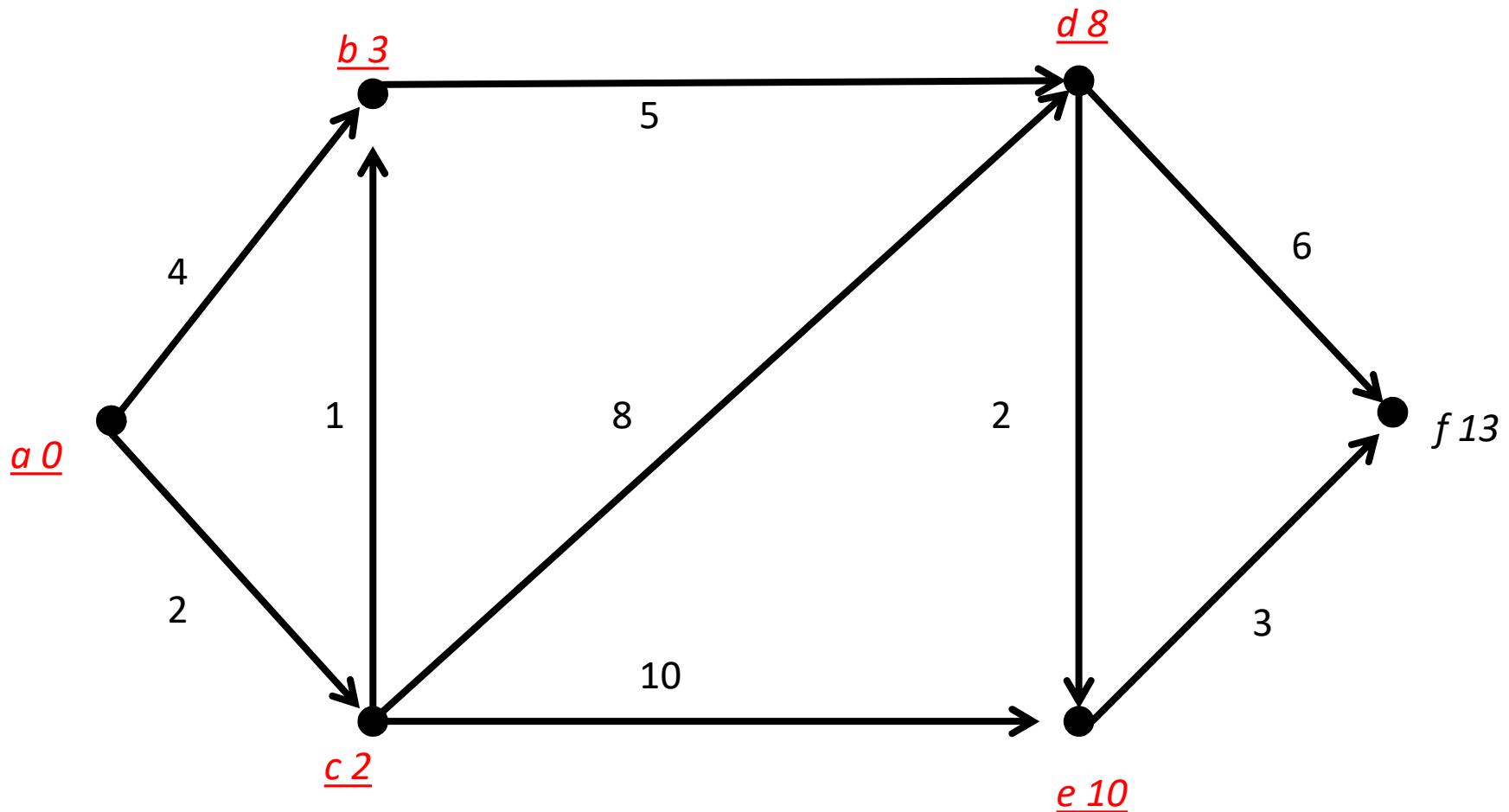
$u = d$

Actualiza a e con $D(e) = 10$ y a f con $D(f) = 14$. $P(e) = d$ y $P(f) = d$.

$S = \{a, b, c, d\}$.

En la siguiente iteración el mínimo de D será e .

Traza del algoritmo de Dijkstra saliendo del vértice a (7 de 8)



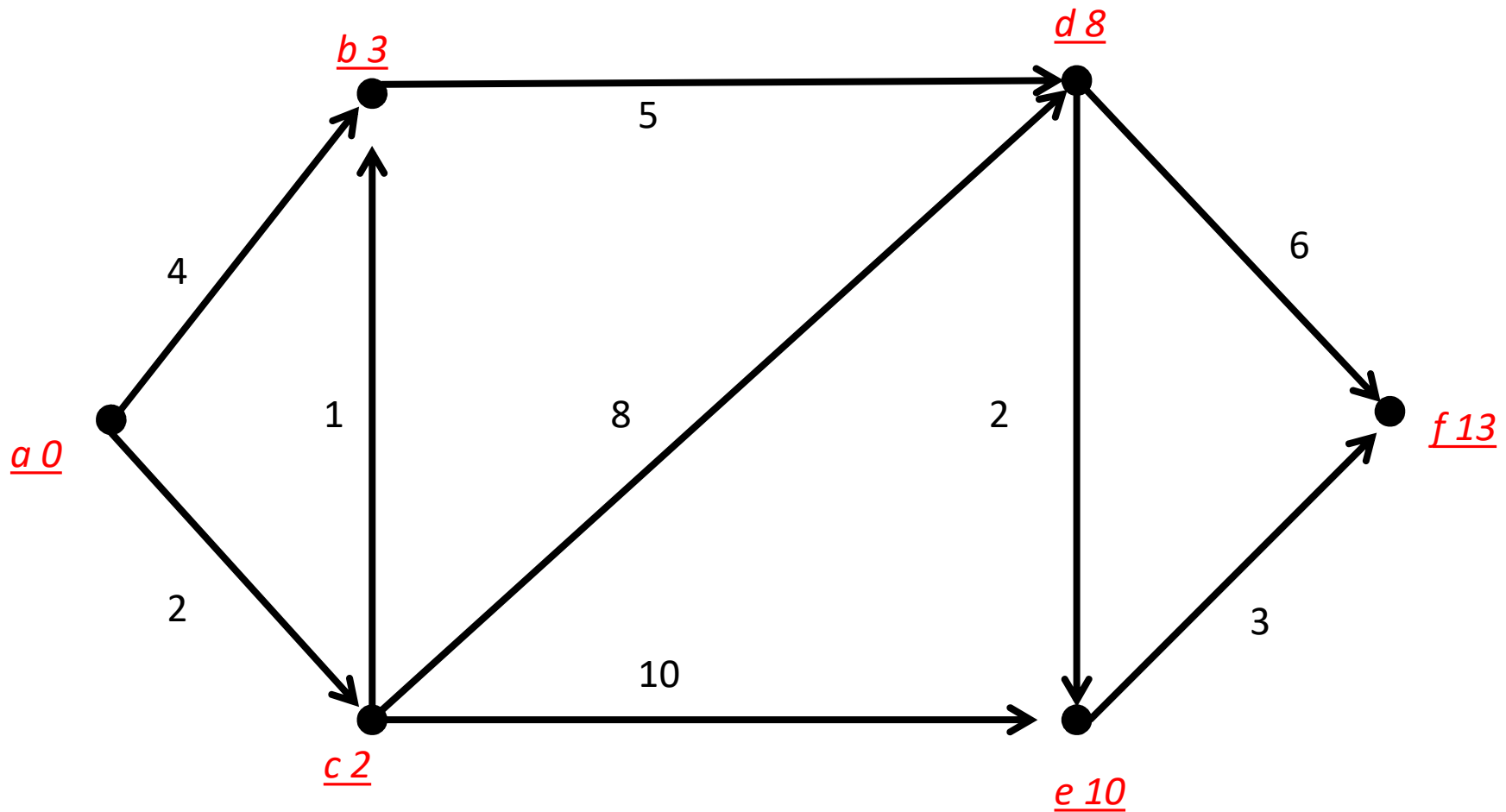
$u = e$

Actualiza a f con $D(f) = 13$. $P(e) = d$ y $P(f) = e$.

$S = \{a, b, c, d, e\}$.

En la siguiente iteración el mínimo de D será f .

Traza del algoritmo de Dijkstra saliendo del vértice a (8 de 8)



$u = f$

Como f no puede alcanzar a ningún vértice, no se actualiza ninguno.

$S = \{a, b, c, d, e, f\}$.

Esta es la iteración final.

Recuperación del camino en Dijkstra

A partir del mapeo P , dado un vértice v destino, el camino de costo mínimo de la fuente a v es $D(v)$ y el camino en orden inverso es $v, P(v), P(P(v)), P(P(P(v))), \dots$

Algoritmo recuperar(P , destino, a)

```
cola ← new Cola()  
cola.enqueue(  $a$  ) { encolo la fuente }  
recuperar_aux(  $P$ , destino, cola )  
retornar cola
```

Algoritmo recuperar_aux(P , v , cola)

anterior ← $P(v)$

Si anterior = 0 entonces

{ el camino es directo (xq el anterior es la fuente) y terminamos }

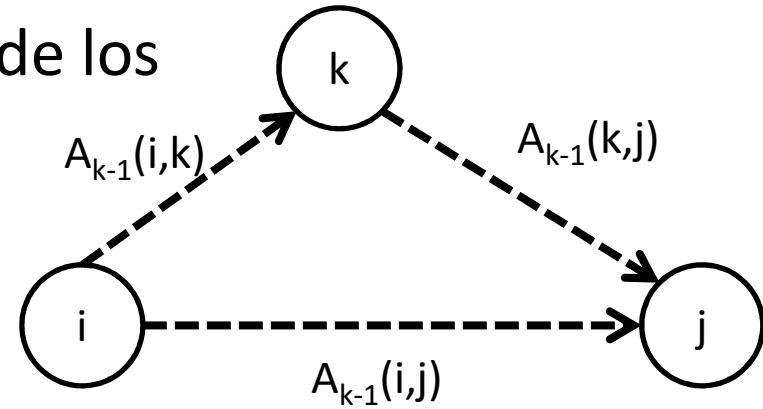
Sino

```
recuperar_aux(  $P$ , anterior, cola )  
cola.enqueue( anterior )
```

Nota: $T_{\text{recuperar}}(n) = O(n)$. Nota: La recuperación se puede plantear en forma iterativa usando un while y una pila.

Algoritmo de Floyd: Caminos mínimos

- Dado un digrafo pesado $G=(V,A)$ donde cada arco tiene un peso numérico no negativo.
- Queremos calcular los caminos de costo mínimo de todos los vértices a todos los vértices.
- Supongamos que $C(i,j)$ es peso del arco (i,j) de A .
- Usaremos una matriz A de $n \times n$ tal que inicialmente $A(i,j)=C(i,j)$, o ∞ si no hay arco entre i y j
- En la iteración k -ésima veremos si actualizamos a A de acuerdo a $A_k(i,j) = \min(A_{k-1}(i,j), A_{k-1}(i,k)+A_{k-1}(k,j))$.
- $P(i,j)$ almacenará a k (i.e., uno de los vértices intermedios en el camino de i a j).

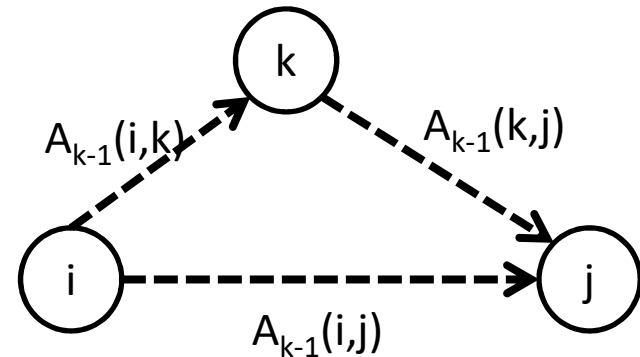


Algoritmo Floyd

Para $i \leftarrow 1..n$ hacer
 para $j \leftarrow 1..n$ hacer
 si hay arco (i,j) entonces $A(i,j) \leftarrow C(i,j)$
 sino $A(i,j) \leftarrow \infty$
 $P(i,j) \leftarrow 0$ { por defecto el camino es directo }

Para $i \leftarrow 1..n$ hacer
 $A(i,i) \leftarrow 0$

Para $k \leftarrow 1..n$ hacer
 para $i \leftarrow 1..n$ hacer
 para $j \leftarrow 1..n$ hacer
 si $a(i,k) + a(k,j) < a(i,j)$ entonces
 $a(i,j) \leftarrow a(i,k) + a(k,j)$
 $p(i,j) \leftarrow k$



$$T_{\text{Floyd}}(n) = O(n^3)$$

Recuperación del camino en Floyd

{ Recupera el camino de i a j y lo almacena en cola que inicialmente debe estar vacía. }

Algoritmo RecuperarCamino(P, i, j, cola)

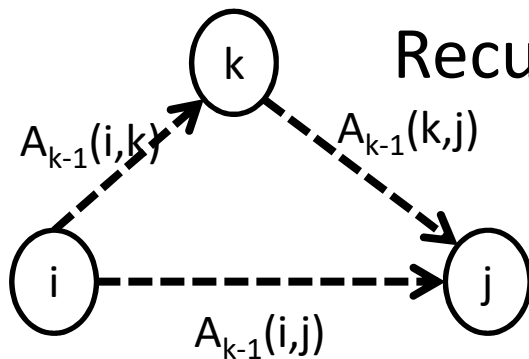
$k \leftarrow P(i, j)$

si $k \neq 0$ entonces

 RecuperarCamino(P, i, k, cola)

 cola.enqueue(k)

 RecuperarCamino(P, k, j, cola)



$T_{\text{RecuperarCamino}}(n) = O(n)$ para un grafo de n vértices.

Bibliografía

- Capítulo 13 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.
- Capítulo 9 de M.A.Weiss, Data Structures and Algorithm Analysis in Java, Third Edition, Pearson, 2012.
- Dijkstra se puede leer también de la páginas 653-661 de la Goodrich, Tamassia y Goldwasser, Data Structures and Algorithms in Java, Sixth Edition. Wiley, 2014.